UGANDA INDUSTRIAL RESEARCH INSTITUTE

Namanve Industrial and Business Park, Mukono District, Uganda

PROGRESS REPORT

DESIGN AND DEVELOPMENT OF A WEB BASED INVENTORY MANAGEMENT SYSTEM FOR UGANDA INDUSTRIAL RESEARCH INSTITUTE (UIRI) NAKAWA AND NAMANVE CAMPUSES

Name	Registration Number	Project Responsibility
Keith Paul Kato	24/U/26593/EVE	Lead reporting, database design and documentation coordination
Komukama Tracy	24/U/06151/EXT	Requirements review, user needs analysis and supporting documentation
Atuhiire Deo Kamate	24/U/14365/PS	System analysis, testing support and validation of database design decisions

Programme	**Bachelor of Science in Computer Science**
Institution	Makerere University, Kampala
Host Organisation	Uganda Industrial Research Institute (UIRI)
Department	Information and Communications Technology (ICT)
Field Supervisor	Mr. William Kakooza, IT Engineer at UIRI
Submitted to	ICT Team, UIRI Namanve
Reporting Period	June-July 2026
Preparation Date	16 July 2026

Project Team and Report Control

This progress report records the current implementation status of the UIRI Inventory Management System. It links the approved proposal, requirements specification, database design, system diagrams, source code and live database evidence to the work that has been completed and the work that remains.

Name	Registration Number	Project Responsibility
Keith Paul Kato	24/U/26593/EVE	Lead reporting, database design and documentation coordination
Komukama Tracy	24/U/06151/EXT	Requirements review, user needs analysis and supporting documentation
Atuhiire Deo Kamate	24/U/14365/PS	System analysis, testing support and validation of database design decisions

Table 1: Project Team and Assigned Contributions

Item	Description
Document Title	Progress Report for the UIRI Inventory Management System
Document Version	1.0
Document Basis	Project brief, proposal, SRS, database design, system design diagrams, source code and live database snapshot
Main Purpose	To report the current state of analysis, design, database implementation, coding, testing evidence, challenges and remaining work
Current Project Status	Working PHP/MySQL prototype with implemented database, inventory, stock, reports, dashboard, audit and security features

Table 3: Document Control

Table of Contents

List of Tables

Abbreviations and Acronyms

Acronym	Full Form
CRUD	Create, Read, Update and Delete
CSRF	Cross Site Request Forgery
CSV	Comma Separated Values
DBMS	Database Management System
ERD	Entity Relationship Diagram
ICT	Information and Communications Technology
IMS	Inventory Management System
PDO	PHP Data Objects
PHP	PHP: Hypertext Preprocessor
RBAC	Role Based Access Control
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
SRS	Software Requirements Specification
UAT	User Acceptance Testing
UIRI	Uganda Industrial Research Institute
XAMPP	Cross Platform, Apache, MariaDB, PHP and Perl

Abstract

This progress report presents the current state of the internship project titled Design and Development of a Web Based Inventory Management System for Uganda Industrial Research Institute (UIRI). The report follows a Makerere-style academic progress-report structure by linking the original project question, proposal, SRS, database design, system design diagrams and current implementation evidence.

The project has progressed from proposal and requirements analysis into a working PHP/MySQL prototype. The implemented system now supports authentication, role based access, branch/campus structure, departments and sections, inventory item registration, item image upload, suppliers, stock-in, stock-out, stock adjustment, requests, transfers, notifications, dashboards, analytics, reports, audit logs and security hardening.

The live database named uiri_ims currently contains 24 tables, including 2 branches, 39 departments, 111 sections/units, 70 categories, 1,043 inventory items, 68 stock transactions, 20 inventory requests, 15 users and 283 audit log entries. These figures show that the database is no longer only a proposed design; it has been implemented and populated with realistic demonstration data for testing and reporting.

Progress Note
Current Assessment: The project is substantially implemented as a prototype. The strongest areas are inventory records, stock movement, dashboard reporting, database population and security. The main remaining work is user acceptance testing, completion of the user manual and final report, improved asset-instance normalization, more serial-number data, SMTP configuration and presentation preparation.

1.0 Introduction and Background

Uganda Industrial Research Institute operates in an environment where ICT equipment, laboratory equipment, engineering tools, office supplies, furniture, machinery, consumables and safety equipment must be recorded, monitored and reported accurately. The original project question required a web based Inventory Management System capable of authentication, inventory management, stock management, supplier management, reporting, search, filtering, database design and security.

The preliminary documents identified the main problem as the difficulty of obtaining a reliable and quick picture of what UIRI owns, where items are located, their current condition and how stock has moved over time. The proposal and SRS therefore emphasized centralized records, secure access, current stock visibility, low stock alerts, supplier transaction history and reports for management decision making.

The implementation has followed the approved database and web application direction, using PHP, MySQL/MariaDB and XAMPP. During coding, the physical schema was adjusted to support a practical prototype with branch-specific categories, section and department hierarchy, item-level asset decision fields and reporting views suitable for demonstration.

2.0 Objectives and Progress Status

Objective	Progress Status	Evidence
Study the inventory problem and identify users, records, reports and rules.	Complete	Proposal and SRS define problem, stakeholders, scope, user classes, functional requirements and acceptance criteria.
Design a normalized relational database.	Complete with implementation refinement	Database design exists; SQL schema and live MySQL database contain users, roles, branches, sections, departments, categories, suppliers, inventory_items, stock_transactions and audit tables.
Implement secure authentication and role-based access control.	Implemented	Login, roles, permissions, session controls, password hashing, account lockout, rate limiting and audit logging are present in code and database.
Enable inventory item registration, editing, classification and tracking.	Implemented	Item CRUD, categories, images, supplier link, serial number, asset code, brand/model, status, condition, funding source and storage location fields are implemented.
Support stock receipt, issue, transfer, return and low-stock alerts.	Substantially implemented	Stock-in, stock-out, stock adjustment, transaction history, transfer workflow and low-stock notifications exist. Return handling is not yet a separate visible workflow.
Provide supplier management and supplier transaction history.	Implemented	Supplier CRUD is implemented; supplier-based filtering is available in inventory and reports.
Generate dashboard summaries and reports.	Implemented	Dashboard, analytics workspace, printable reports, low-stock report, valuation report, movement report and CSV exports are implemented.
Enforce validation, password protection, sessions and audit logs.	Implemented	CSRF helper, prepared SQL statements, clean output helper, security headers, login history, rate limits and audit_log are implemented.

Table 4: Objectives and Progress Status

3.0 Methodology and Technical Approach

The project followed an incremental database-driven approach. The first stage involved problem analysis and requirements documentation. The second stage converted requirements into a database design, data dictionary and system design diagrams. The third stage implemented the application using PHP pages, shared configuration and helper functions, MySQL tables and XAMPP for local development.

- Requirements analysis was guided by the ICT Team project question, the system proposal and the SRS.
- Database work used a relational MySQL/MariaDB design with primary keys, foreign keys, indexes and controlled status values.
- Implementation used procedural PHP modules organized into pages, shared includes, upload folders, migrations and SQL seed files.
- Security work used password hashing, prepared statements, session hardening, CSRF tokens, security headers, login history, rate limiting and audit logging.
- Testing evidence was collected from the live uiri_ims database, implemented pages, migrations and source-code inspection.

4.0 Progress Against Deliverables

Deliverable	Current Status	Progress Note
System Proposal	Complete	Located in documentation/proposal and final_report. Defines problem, objectives, scope, methodology and expected deliverables.
System Requirements Specification	Complete	Located in documentation/srs. Defines functional, non-functional, database and acceptance requirements.
Database Design, ERD and Data Dictionary	Complete	Located in documentation/database_design. Current implementation refined some table names and combined some asset/stock fields for prototype delivery.
System Design Diagrams	Complete	Located in documentation/system_design. Includes use case, activity and class design coverage.
Source Code	Substantially complete	PHP pages, includes, assets, migrations, SQL seed files and upload support are present.
Fully Functional Inventory Management System	Working prototype	Core modules run against the live uiri_ims database. Additional data cleanup and UAT remain.
User Manual	Pending	Needs to be written after screens and workflows stabilize.
Final Project Report	Pending/In progress	This progress report prepares the implementation evidence needed for the final report.
Presentation and Demonstration	Pending	Database and system screens are ready for demonstration; presentation slides still need final assembly.

Table 5: Progress Against Deliverables

5.0 Database Design and Implementation Progress

The original database design proposed a strongly normalized schema with separate entities for campuses, departments, locations, inventory categories, units of measure, item types, inventory items, suppliers, supplier_items, acquisition_batches, asset_statuses, asset_instances, stock_balances, stock_transactions, requisitions, maintenance_records, audit_logs and report_logs.

The implemented database keeps the same core intention but uses a practical prototype structure. For example, campuses are represented as branches, inventory locations are partly represented through section, department and storage_location fields, while current stock and key asset decision fields are stored directly on inventory_items. This makes the prototype easier to demonstrate while still preserving transaction history through stock_transactions.

5.1 Live Database Snapshot

Database Area	Current Evidence
Database name	uiri_ims
Core table count	24 tables
Branches/campuses	2: UIRI Nakawa and UIRI Namanve
Departments and sections/units	39 sections and 111 departments/units
Categories	70 category records: 33 for Nakawa and 37 for Namanve
Inventory items	1,043 active inventory records
Stock units and value	8,484 units with a current stock value of UGX 21,205,901,200
Stock movement history	68 stock transactions: 42 stock-in, 22 stock-out, 2 transfer-in and 2 transfer-out
Low-stock exposure	425 items at or below the minimum stock threshold
Users and roles	15 users across 6 roles
Audit evidence	283 audit log entries and 90 login history records
Security evidence	rate_limits table exists; current rate limit count is 0 at snapshot time

Table 6: Live Database Snapshot

5.2 Implemented Schema Groups

Schema Group	Implemented Tables	Purpose
Access control	users, roles, permissions, role_permissions, user_permissions, login_history, rate_limits	Controls login, roles, permissions, sign-in monitoring and brute-force protection.
Organizational structure	branches, sections, departments	Represents UIRI Nakawa/Namanve and their departments or units.
Inventory master data	categories, suppliers, inventory_items	Stores item classifications, suppliers and item records with asset and stock fields.
Stock workflow	stock_transactions, inventory_requests, transfers, transfer_items	Records stock movement, request approvals and inter-branch transfer workflow.
Reporting and operations	reports, notifications, settings, audit_log	Supports report metadata, alerts, application settings and accountability logs.
Extended workflow	procurement_requests, purchase_orders, goods_received_notes, equipment_maintenance	Adds procurement and maintenance support beyond the minimum project brief.

Table 7: Implemented Schema Groups

5.3 Implementation Changes from Preliminary Design

Preliminary Design Idea	Current Implementation	Progress Report Interpretation
campuses table	branches table	The naming changed, but the same concept is implemented for Nakawa and Namanve.
Separate locations table	sections, departments and inventory_items.storage_location	The prototype captures organizational placement and storage location without a full location master table.
asset_instances table	asset fields on inventory_items, including asset_code, serial_number, asset_status and asset_condition	Acceptable for prototype; future version should separate individual asset instances for stronger normalization.
stock_balances table	inventory_items.current_stock and stock_transactions history	Current stock is fast to read, while movement history is still preserved. A future stock_balances table can improve multi-location quantities.
report_logs table	audit_log records report generation; reports table exists but has no saved rows yet	Report access is audited, but saved report metadata is still incomplete.
Chart.js graphs	CSS/SVG dashboard visuals and tabular reports	Management visuals are present; dedicated Chart.js graphs remain optional enhancement.
Executive role support	Code references Administrator/Executive, but live roles table currently has six roles and no Executive role	Migration exists; live DB should be aligned before final demonstration if Executive view is required.

Table 8: Implementation Changes from Preliminary Design

6.0 Functional Progress Against the Project Brief

Project Requirement	Current Progress	Evidence From Repository and Database
User authentication and authorization	Implemented	Login, registration, password hashing, roles, permissions, admin user management, profile photos, login history and security controls are present.
Inventory management	Implemented	items.php supports item management, categorization, image upload, asset code, serial number, brand/model, asset status, condition and print views.
Stock management	Substantially implemented	stock_in.php, stock_out.php, stock_adjustment.php and transactions.php record stock movement and update current stock. Low-stock notifications are implemented.
Supplier management	Implemented	suppliers.php supports supplier add, edit, status toggle and deletion safeguards; reports can filter by supplier.
Reporting dashboard	Implemented	dashboard.php and analytics.php provide summary KPIs, stock risk, request queues, movement visuals, supplier exposure, campus comparison and decision signals.
Monthly and annual reports	Partly implemented	reports.php supports date filters, movement report, valuation, low-stock report, print mode and CSV export. Annual views can be produced through date range filters.
Search and filtering	Implemented	Items, reports, transactions, login history and notifications include filters and search fields.
Database design	Implemented	database.sql, migrations and live uiri_ims database are available. Data dictionary exists in documentation.
Security features	Implemented	Input validation, prepared statements, CSRF helper, output escaping, session hardening, email verification, rate limiting, security headers and audit logging are present.

Table 9: Functional Progress Against the Project Brief

7.0 Testing and Validation Evidence

Testing at this progress stage focused on verifying that the database exists, implemented tables can be queried, and source modules correspond to the requirements in the SRS and project brief. The live database was queried through XAMPP PHP using the same root credentials defined in includes/config.php.

Test Area	Evidence	Current Result
Database connection	PDO connection to localhost/uiri_ims	Successful
Schema completeness	SHOW TABLES and table counts	24 tables present
Inventory data	COUNT inventory_items and stock summary	1,043 items and 8,484 units
Stock transaction history	Grouped stock_transactions by transaction_type	Stock-in, stock-out and transfer records exist
Access control data	Roles, users and permissions tables queried	6 roles, 15 users and 34 role-permission mappings
Audit trail	audit_log count and login_history count	283 audit entries and 90 login history records
Report functions	Source inspection of reports.php, dashboard.php and analytics.php	Reports, filters, print views and CSV export are implemented

Table 10: Testing and Validation Evidence

Progress Note
Testing Gap: Formal user acceptance testing, browser-by-browser screenshots, complete security test logs and a final checklist still need to be added before the final report and demonstration.

8.0 Challenges, Changes and Mitigation

Challenge or Change	Effect on Project	Mitigation or Recommendation
Preliminary database design was more normalized than the working prototype.	Some planned entities, such as asset_instances and stock_balances, are not separate live tables.	Document the prototype decision and add these tables in a future refinement if individual asset tracking becomes the main assessment focus.
Real UIRI inventory data may be incomplete or sensitive.	Demonstration data has to be used for some screens and reports.	Continue using realistic dummy/demo records while keeping real institutional data protected.
Many items lack serial numbers or purchase dates.	Reports on acquisition year, warranty and asset aging are less complete.	Prioritize data cleaning for serial_number, purchase_date, brand_model, asset_condition and storage_location.
Transfers and procurement structures exist but live transfer/procurement records are empty.	Demonstration may not show those workflows with real examples.	Seed a small number of approved demonstration transfers and procurement records before presentation.
SMTP/PHPMailer setup may not be fully configured.	Email verification and reset links may depend on fallback display during local demo.	Run composer install if needed and configure SMTP in settings before final deployment.
Reports table exists but report rows are not saved.	Report generation is audited, but saved report metadata is incomplete.	Either populate reports table on report generation or explain that audit_log currently provides accountability.

Table 11: Challenges, Changes and Mitigation

9.0 Remaining Work Plan

Task	Priority	Expected Output
Complete user manual with login, item management, stock in/out, reports and admin workflows.	High	User manual ready for submission.
Run end-to-end module testing for authentication, item CRUD, stock in, stock out, adjustment, reports and audit logs.	High	Test results table and screenshots for final report.
Clean demonstration data by adding serial numbers, purchase dates, item images and asset conditions.	High	Stronger report evidence for computers, equipment and asset status.
Align live roles with code, especially the optional Executive role if needed.	Medium	Consistent role list before demonstration.
Seed transfer, maintenance and procurement demonstration records.	Medium	Complete workflow examples for presentation.
Decide whether to implement separate asset_instances and stock_balances now or document them as future enhancements.	High	Clear database-design explanation for assessors.
Configure SMTP/PHPMailer or prepare fallback demonstration steps.	Medium	Reliable email verification and password reset demo.
Prepare final report and presentation slides.	High	Final submission package and demonstration flow.

Table 12: Remaining Work Plan

10.0 Conclusion

The UIRI Inventory Management System has moved from requirements analysis and database design into a working database-backed PHP prototype. The current system already satisfies most of the core project requirements: secure access, inventory item management, supplier management, stock movement, reporting, dashboard summaries, search/filtering, low-stock monitoring and audit logging.

The most important academic point to document in the final report is the difference between the preliminary ideal database design and the implemented prototype schema. The implementation remains suitable for demonstration, but the final report should clearly explain that some highly normalized asset and stock-balance entities were simplified into inventory_items for the prototype. With additional testing, data cleaning and user documentation, the project will be ready for final assessment and demonstration

11.0 References

[1] ICT Team, UIRI Namanve, Internship Project Question: Design and Development of an Inventory Management System for Uganda Industrial Research Institute, 2026.

[2] K. P. Kato, K. Tracy and A. D. Kamate, System Proposal: Design and Development of a Web Based Inventory Management System for UIRI, 2026.

[3] K. P. Kato, K. Tracy and A. D. Kamate, Software Requirements Specification for the UIRI Inventory Management System, 2026.

[4] K. P. Kato, K. Tracy and A. D. Kamate, Database Design Document and Data Dictionary for the UIRI Inventory Management System, 2026.

[5] K. P. Kato, K. Tracy and A. D. Kamate, System Design Diagrams for the UIRI Inventory Management System, 2026.

[6] UIRI IMS repository source code, database.sql, migrations, SECURITY_IMPROVEMENTS.md and live uiri_ims database snapshot, accessed 16 July 2026.

[7] T. M. Connolly and C. E. Begg, Database Systems: A Practical Approach to Design, Implementation, and Management, 4th ed., Pearson Education, 2005.

[8] IEEE Computer Society, IEEE Recommended Practice for Software Requirements Specifications, IEEE Std 830-1998, 1998.

Appendix A: Live Database Snapshot

Table	Rows	Table	Rows
audit_log	283	branches	2
categories	70	departments	111
equipment_maintenance	3	goods_received_notes	0
inventory_items	1,043	inventory_requests	20
login_history	90	notifications	11
permissions	16	procurement_requests	0
purchase_orders	0	rate_limits	0
reports	0	role_permissions	34
roles	6	sections	39
settings	4	stock_transactions	68
suppliers	13	transfer_items	0
transfers	0	user_permissions	0
users	15		

Table 13: Live Database Table Counts

Snapshot source: live uiri_ims database queried through XAMPP PHP on 16 July 2026. The snapshot should be regenerated before final submission if new records are added.